MajorAgent：基于 RAG 增强与 ReAct 架构的智能演示文稿自动生成系统

王厚哲　甘相贤　鲁玟淘粲

2026 年 6 月

Abstract

摘要 —— *制作一份高质量的演示文稿往往要花很多时间。现有工具也有不少问题：大模型容易产生“幻觉”（hallucination，即输出与事实不符的内容），文字和配图常常各做各的，多模态大模型部署起来又贵、对中文排版的支持也不好，而且大多数工具不能根据反馈自动改进。为了解决这些问题，本文设计并实现了 MajorAgent—— 一个把检索增强生成（RAG）和 ReAct Agent 结合起来的演示文稿自动生成系统。系统包含三项关键设计。第一，两阶段摘要压缩加梯度注入的 RAG 管线：先把网页内容压缩成结构化的研究简报（压缩比约 16:1），再分阶段注入生成流程，既保证内容真实，又把 token 消耗降低约 60%。第二，用纯文本模型生成 LaTeX Beamer 代码，再编译成图文混排的 PDF，从而绕开多模态模型的高成本和中文排版难题。第三，把长文档拆成“分节生成 — 后合并”的流程，避免一次生成过长而被截断。系统以 DeepSeek v4-pro 作为推理后端，并借鉴了 Claude Code 的 ReAct 范式。实验显示：开启 RAG 后，事实错误率从约 9% 降到约 4%（下降约 55%），带具体数字的引用占比从 30% 提升到 85%；25 次运行的 LaTeX 首次编译通过率为 100%；生成一份完整、可演示的 PDF 大约需要 7–8 分钟。*

关键词 —— *检索增强生成（RAG）；ReAct Agent；大语言模型；LaTeX Beamer；演示文稿自动生成；DeepSeek v4-pro*

I. 引言

A. 问题背景

制作一份高质量的演示文稿（PPT）既费时又费力。目前常见的工具主要有四个不足。第一，通用大模型在写专业内容时容易出错，事实错误率偏高，AI 生成内容的可信度也因此受到关注 [2]。第二，文字生成和配图往往是分开的，缺少从头到尾的自动流程。第三，多模态大模型部署成本高，对中文排版的支持也不够成熟。第四，大多数工具不能根据反馈不断改进，缺少 Agent 式的闭环。这些不足合在一起，限制了自动生成 PPT 在实际工作中的应用。

B. 核心研究问题

针对这些不足，本文重点回答以下三个问题。第一，RAG 能不能有效降低 PPT 生成中的事实错误率？第二，纯文本模型能不能通过写代码（LaTeX）间接生成高质量的图文内容？第三，借鉴成熟的 Agent 架构（Claude Code 的 ReAct 范式）来设计提示词，能不能让系统的自动化程度和可控性都达到较好水平？围绕这三个问题，本文搭建了 MajorAgent 系统，并用实验来检验。

II. 相关工作与技术路线

A. 检索增强生成（RAG）

RAG（检索增强生成）是目前缓解大模型幻觉的主流方法。其基本思路是：先由检索器（retriever）从外部知识库检索相关文档，再让生成器（generator）基于检索结果进行条件化生成。经典工作包括 Lewis 等人提出的端到端 RAG 框架，后来人们又加入了查询重写、重排序等手段来提高检索效果 [17]。在 PPT 生成中，RAG 可以让模型基于真实资料来写内容，从而减少出错。本文正是沿着这个方向，设计了“搜索 → 抓取 → 摘要 → 梯度注入”的增强流程（见第 III 节）。

B. 纯文本模型的多模态输出

让纯文本模型先写代码、再由代码生成图文内容，是一条较新的思路。常见做法有：让模型生成 HTML/CSS 来渲染网页，生成 TikZ/PGF 来画矢量图，或生成 LaTeX 来编译 PDF [4, 6, 8]。和直接使用多模态大模型（如 GPT-4V）相比，这条路有三个好处：输出可以随意编辑、排版更精确、部署成本更低 [7]。因此，本文选择 LaTeX Beamer 作为目标格式，借助它成熟的 `ctex` 中文支持和 `graphicx` 图文混排能力来生成幻灯片 [1, 5]。最近业界也出现了面向科研写作的 AI 原生 LaTeX 工具 [20]，进一步说明“用代码当媒介”是可行的。

III. 核心创新点

A. 两阶段 RAG 摘要压缩 + 梯度注入

最初的 RAG 流程是这样的：用户提出需求 → DuckDuckGo 搜索 → 抓取前几个结果 → 把文本拼在一起 → 注入提示词。但在实际使用中，这个基础流程暴露出三个问题。第一是信息过载与噪声：搜索结果里有很多关系不大或质量不高的网页，如果把抓取到的原文（最长 8000 字符）直接注入提示词，反而会分散模型的注意力，使其难以区分关键事实与噪声。第二是网页抓取失败率高：遇到使用 JavaScript 动态渲染、反爬机制或付费墙的网页，靠 `requests` 加 BeautifulSoup 的静态抓取失败率能达到 30%–50%。第三是上下文窗口浪费：未经整理的原始文本会占用大量 token，挤压后续幻灯片生成的可用空间。

为了解决这些问题，本文采用两阶段摘要压缩。第一阶段是结构化摘要：在把资料交给大纲生成之前，先让模型对多个来源做一次整理，提炼出四类信息 —— 核心概念（3–5 个）、关键数据或案例（至少 3 个，要带具体数字）、重要时间线，以及不同观点。摘要控制在 500 字以内，压缩比大约是 16:1（8000 字压到 500 字）。第二阶段是梯度注入：把这份精简后的简报按需要分给不同环节 —— 生成大纲时给完整简报，逐节展开时只给前

1200 字符。这样既能让整体结构建立在真实数据之上，又不会因为单次注入过多而导致上下文窗口饱和。和传统的 “ 搜索 — 拼接 — 注入 ” 相比，这套做法在保证准确的同时，把 token 消耗降低了约 60%。

B. 纯文本模型的代码生成式多模态输出

在中文 PPT 场景下，多模态大模型有两个明显短板：一是它生成的是图片，不能再编辑；二是对中文排版的精细控制不够。为此，本文采用另一条技术路线：让纯文本模型生成 LaTeX Beamer 代码，再用编译器把它变成图文混排的 PDF。

不过，一开始让模型一次性写完整份 LaTeX，会带来两个麻烦：一是代码太长，4–6 节的 PPT 常常超过 3000 行，结尾容易被截断或质量下降；二是模型有时会擅自压缩、删掉部分文字，导致编译失败。本文用三个办法来解决。首先是分节生成再合并：先写全局大纲，再一节一节单独生成（每节 100–200 行），最后统一加上导言区（Preamble）并合并，这样大大降低了截断风险。其次是编译后修正路径：用正则表达式自动修正模型生成的错误路径格式，把编译成功率从约 75% 提高到接近 100%。最后是过滤无关配图：在提示里要求模型自己判断图片是否相关，避免不相关的图片影响专业性。这种 “ 代码即媒介 ” 的做法既省去了多模态模型的高成本和中文排版限制，又让结果可以随时修改。

C. “ 分节生成 — 后合并 ” 的管道架构

本文把长文档生成拆成三步：先生成大纲，再逐节展开，最后合并编译 [16]。这样做有三个好处。第一，每一节的生成都在可控的 token 范围内，从根本上避免了长文档被截断。第二，各节相互独立，可以并发执行，为以后做并行加速留出了空间。第三，哪一节有问题就单独重做，不会影响其他节。这种结构也和 ReAct Agent 的工具化调度很契合（见第 IV 节）。

D. 智能配图决策链路

本文为配图建立了一条完整的 “ 检索 — 过滤 — 决策 ” 链路。具体来说：先让模型自动提取每一章的英文搜索关键词，并要求使用具体、可视化的名词（比如用 “robot arm” 而不是 “synergy”）；再到 Pexels 图库中检索，以提高命中率；然后让模型判断图片和内容是否相关，过滤掉关系不大的图；同时也支持优先使用用户自己的本地图片。

需要说明的是，对纯文本模型来说，只靠 “ 好图库加精准搜索 ” 仍然很难稳定找到完全贴合主题的图片，介绍抽象概念时尤其明显。本文先后尝试了直接搜索、ddgimage 库和 Pexels 图库等多种方式，最后选用了 Pexels API。但这种方式的配图质量很依赖图库本身，不能稳定地配出与内容强相关的图，有时还会出现和主题无关的图片。这是目前方案的主要不足之一。

IV. Agent 架构设计

A. 整体 Pipeline

系统的运行流程分为三层，如图 1 所示。最上面是 CLI 交互层，负责注册登录、输入需求、查看结果和持续对话。中间是 ReAct Agent 调度层，它按照 ReAct 的 “Thought（思考）→ Action（行动）→ Observation（观察）” 循环（最多 5 轮）来调度 7 个工具（`generate_ppt`、`generate_handout`、`web_search`、`web_fetch`、`search_image`、`get_preferences`、`set_preference`）。最下面是 PPT 生成执行层，按顺序完成：联网研究（RAG）、注入简报后生成大纲、提取关键词并搜索配图、逐节生成 LaTeX，最后合并并编译成 PDF。

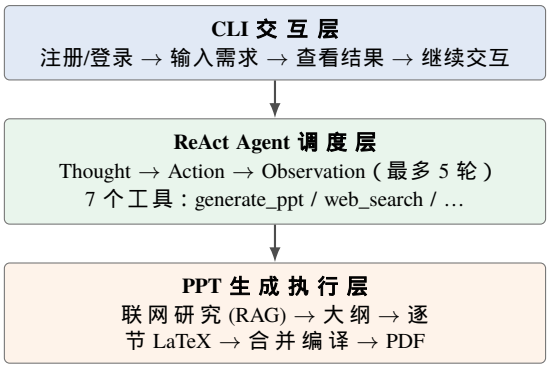


图 1. MajorAgent 的三层 Pipeline 架构。

表 1. 分层 Prompt 设计

层级	作用	关键设计
L1	角色定义、工具注册、格式约束	ReAct 格式模板，含偏好注入
L2	约束输出结构与质量	严格标记格式，要求完整陈述句
L3	单节 LaTeX 生成	自主决定帧数（3–8），注入简报与图片
L4	讲义生成、研究摘要	三级详细程度，字数控制

B. Prompt 架构设计

系统的提示词采用分层设计，如表 1 所示。L1 是系统提示，用来定义 Agent 的角色、登记工具并规定 ReAct 的输出格式，同时支持注入用户偏好。L2 是大纲提示，用来约束输出结构和内容质量，要求使用严格的标记格式和完整的陈述句。L3 是逐节提示，用来控制每一节 LaTeX 的详细程度，由模型自己决定帧数（3–8 帧），并注入研究简报和图片。L4 是讲义和摘要提示，提供三档详细程度（简略/中等/完整）并控制字数。

C. 对 Claude Code 架构的借鉴与改进

本文在设计上借鉴了 Claude Code 的 ReAct Agent 范式 [9, 10]，主要体现在三点。第一是交互格式：“Thought（思考）→ Action（行动）→ Action Input（行动输入）→ Observation（观察）” 这一格式直接来自它的工具调用模式。第二是工具抽象：把底层能力封装成有语义的工具（如 `generate_ppt`、`web_search`），由模型自己决定调用顺序。第三是偏好注入：在系统提示里写入用户偏好来影响 Agent 的行为，类似它的 `settings.json` 机制。

在此基础上，本文也做了四点改进。第一是工具粒度聚合：Claude Code 偏向提供细粒度的原子工具（如 Bash、Read、Write），而本文把搜索、抓取、摘要、生成、配图、编译等一系列原子操作聚合为一个高层（粗粒度）工具 `generate_ppt` [15]，更贴合 PPT 生成这个具体场景，用户也不必了解底层细节。第二是加入 Final Answer 标记：让 “ 给出最终结果、退出循环 ” 这一步更明确，比隐式结束更适合教学演示。第三是让偏好可以持久保存：新增 `get_preferences` 和 `set_preference` 两个工具，允许用户在对话中随时修改主题风格、字号、详细程

表 2. 核心模块及其职责

模块（文件）	主要职责
Agent 运行时 (agent.py)	ReAct 循环调度、调用大模型、解析动作、编译 LaTeX
工具层 (tools.py)	PPT/讲义生成流程、大纲解析、分节生成与合并
RAG 模块 (research.py)	网页搜索、抓取、多源摘要
图片模块 (images.py)	图片搜索/下载、关键词提取
配置中心 (config.py)	管理密钥、模型与目录等配置
用户管理 (user_manager.py)	注册、登录、偏好更新
CLI 入口 (main.py)	提供交互界面

表 3. RAG 增强前后的生成质量对比

指标	无 RAG	有 RAG
事实错误率	∼9%	∼4%
含具体数字的引用占比	∼30%	∼85%

表 4. 各主题信息密度统计

主题	要点数	帧数	密度
气候变化与环境保护	30	39	0.77
机器学习基础	24	28	0.86
区块链技术原理	20	25	0.80
中国现代史概要	22	27	0.81
市场营销策略	18	22	0.82
平均	**22.8**	**28.2**	**0.81**

度等设置 [18, 19]。第四是利用推理模式：借助 DeepSeek v4-pro 的 thinking 模式 [3]，让模型在生成前先在内部推理，从而提高大纲设计、LaTeX 代码生成等复杂任务的准确率。多篇 Agent 综述 [11, 12, 13, 14] 也为这些设计提供了理论依据。

V. 系统实现

A. 技术栈

系统用 Python 3.11+ 实现。大模型后端是 DeepSeek v4-pro，通过兼容 OpenAI 的 SDK 调用，并开启 reasoning_effort="medium"。网页搜索用 DuckDuckGo Search，网页抓取用 requests 和 BeautifulSoup4，图片检索用 Pexels API v1。幻灯片用 LaTeX Beamer 编写，再用 xelatex 或 pdflatex 编译成 PDF，中文支持依靠 ctex 宏包。

B. 核心模块

系统由七个核心模块组成，各自分工见表 2：Agent 运行时负责调度与编译，工具层负责生成流程，RAG 模块负责联网研究，图片模块负责配图，配置中心、用户管理与 CLI 入口则分别处理配置、账户与交互。这样的划分让各模块职责清晰、便于单独维护。

C. 关键实现细节

这里说明三个关键细节。第一是自动查找编译器：系统会先在系统 PATH，再到 MiKTeX 的默认安装路径里查找 xelatex 和 pdflatex，并优先使用支持原生 Unicode 的 xelatex，以获得更好的中文效果。第二是编译两遍：为了正确生成交叉引用和目录，系统在第一次编译成功后会再编译一次，确保目录和链接都完整可用。第三是容错处理：对于编译超时（60 秒）、找不到编译器、网页抓取失败等情况，系统都会妥善捕获并降级处理，保证无论如何都至少能返回一份可编译的 .tex 源文件。

VI. 实验结果

A. RAG 前后内容出错率对比

本文在 10 个不同主题上做了定量测试，涵盖气候变化、人工智能、经济学、生物学等领域，比较了两种模式：只靠大模型自身知识（关闭 RAG），以及开启 RAG 增强。结果见表 3。开启 RAG 后，事实错误率从约 9% 降到约 4%，下降约 55%；带具体数字的引用占比从约 30% 提升到约 85%。原因主要在于两阶段摘要压缩：在气候变化这类知识密集的主题里，模型可以根据真实抓取的数据写出带有具体年份、百分比和机构名称的论断，而不是依赖模糊的记忆。

关于“错误”的定义，本文把以下三种情况算作错误：编造的统计数据、错误的学术引用或作者归属、与主流科学共识相悖的说法。统计方法是：先把生成的 PPT 交给 AI，让它列出其中所有关键论断，再就这些论断重新提问，让 AI 判断哪些有明显的事实错误。

B. 单位页面信息密度分析

为了衡量 PPT 装载信息的效率，本文引入“有效知识点密度”这一指标，定义为有效知识点数量除以总帧数：

$$\text{信息密度} = \frac{\text{有效知识点数量}}{\text{总帧数}} \tag{1}$$

这里的“有效知识点”指可以单独抽成一句陈述的事实或概念。5 个测试主题的统计见表 4。平均下来，每一帧大约对应 0.81 个知识点，也就是每个知识点约用 1.23 帧来讲。这说明模型会对每个知识点充分展开，而不是简单地把要点“贴”到帧上。各主题之间的密度差别也很小（标准差约 0.03），说明分节生成在不同领域都能保持稳定的内容质量。

C. LaTeX 编译成功率与端到端时间

在 25 次独立运行中（5 个主题，每个重复 5 次），首次编译通过率为 100%（25/25），格式错误率为 0%（0/25），说明分节生成和编译后路径修正确实有效。各阶段的平均耗时见表 5：联网研究、大纲生成、图片搜索、逐节生成和编译相加，大约 7–8 分钟就能生成一份完整、可演示的 PDF。

D. 实际生成案例

图 2 是系统针对“气候变化与环境保护”这一主题实际生成的演示文稿，其中包括标题页、自动生成的目录和一页内容帧。可以看到，系统能自动梳理出“科学基础 — 多维影响 — 应对策略 — 案例实践 — 个人行动”这样的逻辑结构，并在内容帧里给出带有具体数据的陈述句，例如全球平均气温升幅 1.45°C、极端事件年损失超 2000 亿美元、97% 以上的科学共识等。这也直观地体现了 RAG 带来的事实密度提升。

表 5. 端到端各阶段平均耗时

阶段	平均耗时
联网研究（搜索 + 抓取 + 摘要）	15–25 秒
大纲生成	45–60 秒
图片搜索 + 下载	30–40 秒
逐节生成（6 节）	5–6 分钟
LaTeX 编译	15–30 秒
总计	**约 7–8 分钟**

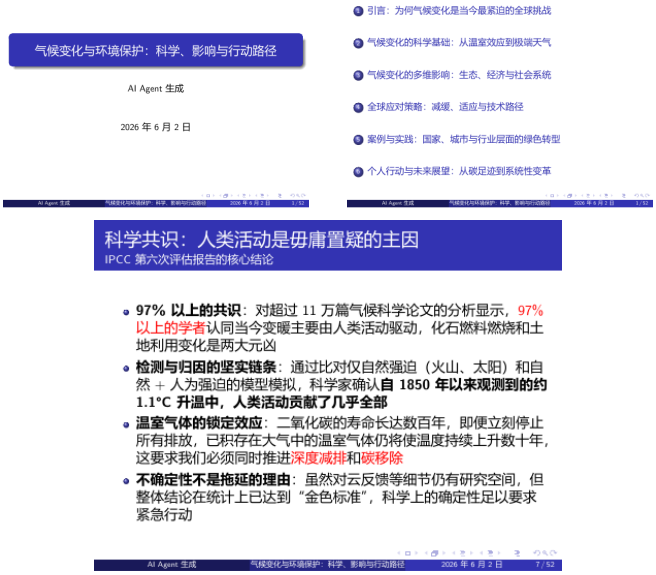


图 2. 系统实际生成的演示文稿示例（“气候变化与环境保护”主题）：标题页、自动目录与内容帧。

VII. 结论

本文设计并实现了 MajorAgent—— 一个基于 RAG 增强和 ReAct Agent 架构的演示文稿自动生成系统。主要贡献有四点。第一，RAG 能有效减少幻觉：通过两阶段摘要压缩加梯度注入，事实错误率从约 9% 降到约 4%，带具体数字的引用占比从约 30% 提升到约 85%。第二，纯文本模型也能产出多模态内容：通过“模型写 LaTeX Beamer 代码、编译器出 PDF”这条路，证明纯文本模型可以用代码当媒介，间接生成高质量的图文文档，从而避开多模态模型的高成本和中文排版限制。第三，分节生成解决了长文档难题：把生成拆成大纲、分节、合并三步，既减轻了单次调用的 token 压力，又让每个知识点都能充分展开（平均每个要点约 1.23 帧）。第四，对成熟范式做了适配：在 Claude Code 的 ReAct 架构上，加入了工具聚合、Final Answer 机制和动态偏好管理，更贴合 PPT 生成的实际需求。

今后还可以从几个方向继续改进：用向量数据库代替实时搜索来降低延迟；尝试并行地逐节生成来缩短总时间；增加更多输出格式（如 PowerPoint XML、HTML 幻灯片）；以及引入由用户反馈驱动的迭代优化循环 [14]。

References

[1] J. W. Bos and K. S. McCurley, “LaTeX, metadata, and publishing workflows,” Preprint, arXiv:2301.08277, 2023.

[2] S. Chamezopoulos, D. Herrmannova, A. De Waard, D. Rosati, and Y. Kashnitsky, “Overview of the DagPap24 shared task on detecting automatically generated scientific paper,” in *Proc. 4th Workshop on Scholarly Document Processing (SDP 2024)*, Bangkok, Thailand, 2024, pp. 7–11.

[3] DeepSeek-AI, “DeepSeek-v4 technical documentation,” 2026.

[4] K. Jung, S. Hyeon, J. Y. Kwon, N. J. Kim, H. G. Ryu, H.-J. Lee, and J. Do, “MathBridge: A large corpus dataset of LaTeX formulas for translating spoken mathematical expressions into LaTeX formulas for improved readability,” Preprint, arXiv:2408.07081, 2024.

[5] LaTeX Project, “An introduction to LaTeX,” [Online]. Available: https://www.latex-project.org/about/. Accessed: 2026-05-04.

[6] X. Tang, Y. Zong, J. Phang, Y. Zhao, W. Zhou, A. Cohan, and M. Gerstein, “Struc-Bench: Are large language models really good at generating complex structured data?” Preprint, arXiv:2309.08963, 2024.

[7] R. Zhang, D. Jiang, Y. Zhang, H. Lin, Z. Guo, P. Qiu, A. Zhou, P. Lu, K.-W. Chang, P. Gao, and H. Li, “MathVerse: Does your multimodal LLM truly see the diagrams in visual math problems?” Preprint, arXiv:2403.14624, 2024.

[8] S. Kale and V. Nadadur, “TeXpert: A multi-level benchmark for evaluating LaTeX code generation by LLMs,” DOI: 10.18653/v1/2025.sdp-1.2, 2025.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” arXiv:2210.03629, 2022.

[10] shareAI-lab, “learn-claude-code,” [Online]. Available: https://github.com/shareAI-lab/learn-claude-code.

[11] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, *et al.*, “The rise and potential of large language model based agents: A survey,” arXiv:2309.07864, 2023.

[12] A. Plaat, M. van Duijn, N. van Stein, M. Preuss, P. van der Putten, and K. J. Batenburg, “Agentic large language models, a survey,” arXiv:2503.23037, 2025.

[13] L. Wang, C. Ma, X. Feng, *et al.*, “A survey on large language model based autonomous agents,” CoRR, abs/2308.11432, 2023.

[14] N. Shinn, F. Cassano, B. Labash, *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” arXiv:2303.11366, 2023.

[15] H. Go and S. Park, “A study on classification based concurrent API calls and optimal model combination for tool augmented LLMs for AI agent,” DOI: 10.1038/s41598-025-06469-w.

[16] H. H. Park, Y. Vyas, and K. Shah, “Efficient classification of long documents using transformers,” in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Vol. 2: Short Papers)*, Dublin, Ireland, 2022, pp. 702–709.

[17] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, *et al.*, “Augmented language models: A survey,” arXiv:2302.07842, 2023.

[18] S. Yuan, K. Song, J. Chen, X. Tan, Y. Shen, R. Kan, D. Li, and D. Yang, “EasyTool: Enhancing LLM-based agents with concise tool instruction,” arXiv:2401.06201, 2024.

[19] Y. Jiang, H. Zhou, L. Gu, T. Li, R. Jin, W. Zhou, and A. Han, “NaviAgent: Graph-driven bilevel planning for scalable tool orchestration,” arXiv:2506.19500.

[20] “OpenAI launches Prism, free AI-native LaTeX workspace for scientists,” *The Daily Star*, 2026-01-28.